

## 1er Obligatorio de Introducción a la Computación Gráfica

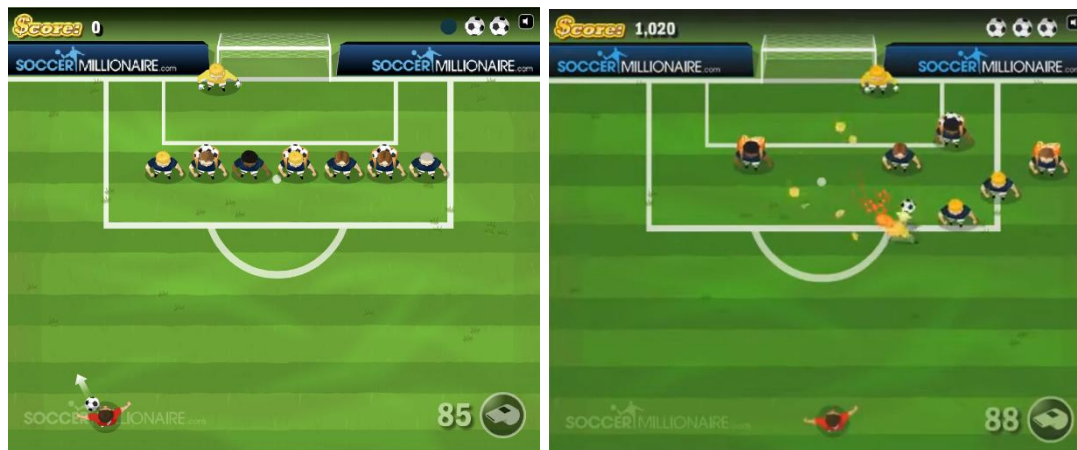
### Videojuego sobre OpenGL

#### Introducción

El obligatorio se centra en conceptos vistos en el curso, así como en las bibliotecas OpenGL y GLU (para efectuar el render), y SDL (para manejo de dispositivos de entrada y creación de ventanas, entre otros conceptos).

Este es un obligatorio de Computación Gráfica, no de diseño de juegos. Se le dará mayor prioridad a todos aquellos aspectos que tengan que ver con el logro de efectos gráficos interesantes y asuntos relativos al "tiempo real", por encima de los aspectos relacionados con la estrategia, las reglas del juego o conceptos no afines a la computación gráfica.

#### Ejercicio (Soccernoid)



Este ejercicio consiste en desarrollar el juego llamado Soccernoid. Es un juego 2D en el que el jugador mueve un personaje o una plataforma para hacer rebotar una bola y destruir bloques o jugadores de fútbol. El juego está basado en Arkanoid pero con una temática de fútbol. El objetivo es eliminar todos los bloques de cada nivel, enfrentando obstáculos y potenciadores que modifican la dinámica del juego, para así avanzar al siguiente desafío. El jugador pierde la partida cuando todas las bolas (pelotas de fútbol) se van de la pantalla.

Video demostrativo del juego y link de interés:

a) Video: <https://www.youtube.com/watch?v=gcXi-e18LQE>

b) Link: <https://www.miniplay.com/game/soccernoid>

No se evaluará el grado de reproducción o copia de los detalles estéticos del juego original. Se evaluará que la aplicación cumpla con los objetivos y características básicas del juego original, así como la innovación resultante del uso de elementos de computación gráfica. **La creatividad, especialmente en los efectos gráficos, influirá positivamente en el puntaje.**

#### Versiones posibles del juego en el pasaje a 3D

Existen diversas posibles variantes del juego 3D. En primer lugar, se podrá realizar una implementación similar a la original, donde la jugabilidad se mantiene en un plano 2D, pero los objetos tienen forma tridimensional. Esta variante permite modificar el ángulo de vista de la cámara e incluso una vista del personaje en primera persona.

Otra variante posible consiste en un mapa que no está en un plano 2D, sino que tiene estructura 3D de tipo cubo. En ese caso la pelota rebota en todas las paredes del cubo y el arco es un rectángulo en la cara opuesta del personaje.

Tanto el personaje principal como los demás jugadores pueden ser realizados de forma simple (como fichas) o compleja (jugadores con movimientos). Recomendamos que comiencen por lo simple. Podría ocurrir que algunos objetos oculten la ubicación de otros, obligando al jugador a cambiar de punto de vista. Se está en libertad de implementar alguna de estas variantes u otra que ustedes crean conveniente, siempre manteniendo el uso de objetos tridimensionales.

En este juego pueden diseñarse múltiples niveles con dificultad progresiva, o bien un único nivel cuya complejidad aumente dinámicamente a medida que el jugador avanza en la partida. **No es obligatorio reproducir la temática de fútbol, pueden generar una representación abstracta tipo Arkanoid tradicional.** En caso de querer reproducir el comportamiento del golero, pueden usar alguna lógica simple como movimiento rectilíneo uniforme de palo a palo, o algo más complejo que dependa de la posición de la pelota.

## Sobre la física del juego

El movimiento del personaje principal no requiere de una física compleja, por lo que debería ser sencillo implementarla. De todas formas, de creerlo conveniente, se podría utilizar algún motor de física, como por ejemplo Bullet, Box2D, Chipmunk, etc. pero su uso deberá estar justificado por un resultado de calidad superior. A su vez, podrían agregarse explosiones con sistema de partículas para representar el choque de la bola con los obstáculos, el festejo de los goles, etc.

## Requerimientos del videojuego

El conjunto de requerimientos que se detallan a continuación representa los mínimos necesarios para aprobar este trabajo obligatorio. La ausencia de alguno de los puntos especificados sin la correcta justificación significará la no aprobación del obligatorio (esto es, la pérdida del curso).

- El programa entregado debe ejecutar sobre el sistema operativo Windows, no debiendo requerir la instalación de ningún driver particular.
- El juego debe estar gobernado por el ratón (para la rotación de la cámara) y las flechas del teclado para el movimiento del personaje o plataforma.
- Es posible rotar la cámara alrededor de la escena o del personaje.
- La tecla V debe permitir el cambio del modo de vista. Modos de vista de ejemplo serían: 1) Vista similar a la vista de la versión original, 2) vista centrada en el personaje (o detrás de él), 3) vista en perspectiva libre donde se pueda ajustar la posición de la cámara y así recorrer la escena.
- Se puede detener el juego (tecla P), y salir (tecla Q).
- **Ajustes (settings):** Se debe disponer de una interfaz para ajustar los siguientes parámetros:
  - *Velocidad del juego:* La velocidad de visualización del juego, la velocidad de los movimientos del gusano y otros elementos debe poder regularse manteniendo aproximadamente constante la cantidad de imágenes generadas por segundo. Para esto se debe multiplicar al tiempo transcurrido entre frames por un valor acorde a la velocidad del juego.
  - *Estados posibles:* wireframe (on/off); texturas (on/off); facetado/interpolado. Cada estado tiene dos valores y se los puede ajustar de forma independiente. Se debe poder cambiar de estado durante la ejecución.
  - *Dirección y color de por lo menos una luz:* las direcciones y colores pueden tener valores predeterminados o se puede implementar una interfaz para poder ajustar sus valores.
- Utilización de al menos una textura (para los personajes, el fondo, objetos, etc.).
- Cargado y renderizado de al menos un modelo 3D (jugador de futbol, arco, adornos, pelota). Para esto se pueden utilizar bibliotecas externas, siempre y cuando sean portables.
- Game HUD básico (puntaje, tiempo, etc.) dibujados con una proyección ortogonal.

- Además de entregar todos los códigos fuentes, se deberá entregar una carpeta que contenga únicamente: el ejecutable, texturas y modelos, bibliotecas utilizadas y archivos XML si los precisan. El ejecutable deberá obligatoriamente funcionar haciendo doble-clic en una máquina Windows. Dicha carpeta no debe exceder los 20 MBytes. Se recomienda almacenar las imágenes en formatos con compresión (JPG, png, gif, etc.). Notar que no se considera los dlls para calcular el peso total.

## Requerimientos opcionales

Los requerimientos que se detallan a continuación representan elementos que tienen un valor agregado, pero no influyen en la aprobación del trabajo obligatorio. La realización de requerimientos opcionales no exime la realización de algún punto especificado como obligatorio.

- Que haya varios niveles.
- Que los objetos de un nivel y sus propiedades (cantidad de bolas, cantidad de jugadores en al barrera, comportamiento del golero) se defina a través de un XML. Se recomienda utilizar alguna de las siguientes bibliotecas:
  - TinyXML (<http://sourceforge.net/projects/tinyxml/>)
  - PugiXml (<http://pugixml.org/>)
- Implementar un editor de niveles.
  - Como resultado se espera que el editor pueda guardar un archivo XML que luego será utilizado como uno de los niveles del juego.
- Implementar efectos gráficos, por ejemplo, utilizar un sistema de partículas para generar explosiones, fuego, lava u otros elementos que pueden estar presente o reaccionar ante las acciones del juego.

## Recursos a utilizar

La sala 315 y las aulas informáticas A,B,C,D disponen de computadoras con Windows. A su vez, en cualquier PC o notebook razonablemente moderno van a poder desarrollar y correr un juego de estas características. Se podrá utilizar OpenGL 1.2 o 1.3, y GLU.

El paquete SDL de desarrollo puede ser descargado desde la página oficial de la librería. Recomendamos utilizar SDL 2.0. <http://www.libsdl.org/>

Se puede utilizar el IDE de Visual Studio. También se pueden utilizar otros compiladores/IDEs C++:

- Mingw dentro del IDE Code::Blocks
- Mingw dentro del IDE Bloodshed Dev-C++

**Desarróllenlo en modo RELEASE** para así acelerar su funcionamiento.

Si no disponen de ninguna máquina Windows, de forma excepcional (previo aviso) les autorizaríamos a utilizar Linux. Los drivers de OpenGL en Windows suelen tener mejor rendimiento que los equivalentes de Linux.

## ¿Trabajo individual o colectivo?

Se deben formar grupos de 3 estudiantes (en algún caso 2 estudiantes, pero con la debida justificación). La formación de los grupos queda por cuenta de los estudiantes. Si fuese necesario, pueden utilizar el foro para buscar compañeros y así poder armar los grupos.

## Defensa del obligatorio (lunes 25 de mayo)

La defensa consta de 2 partes: una presencial y otra en base a la documentación presentada.

- **Defensa presencial:** Se realizará en la sala 315. La lógica de la defensa es que se presenta cada grupo (con todos sus integrantes) y entregan el ejecutable y la documentación (máximo 8 páginas

sin contar la carátula ni el índice). Los grupos realizan una breve demostración de sus aplicaciones, que deben ejecutar preferentemente sobre el sistema operativo Windows. Es importante que prueben sus aplicaciones antes de la defensa en diversas máquinas para evitar contratiempos innecesarios. También podrán comentar algunos aspectos de la aplicación y de la documentación. Los docentes harán preguntas sobre el trabajo realizado.

- **Documentación:** Se hará entrega al docente la documentación que consiste en:
  - Una documentación en la que se explica:
    1. Arquitectura de la aplicación.
    2. Explicar cómo resolvieron el modelado de objetos, los temas de iluminación, textura, y otros aspectos que hacen a los temas gráficos de la aplicación.
    3. Agregar un capítulo de referencias donde se mencione la documentación utilizada. Si se utilizaron partes de código fuente extraído de otras aplicaciones, explicar cuáles fueron y de dónde se extrajeron. Lo mismo para ideas extraídas de otras aplicaciones.
    4. Versión utilizada de SDL.
- **Entrega:** Cada grupo entrega en la defensa un zip con: código fuente, ejecutable, archivo de la documentación. Recuerden (lean más arriba) las características de la carpeta con el ejecutable, modelos y textura.

En base a la evaluación de ambas instancias se definirá la nota final del obligatorio.

Parece obvio, pero no está permitido utilizar códigos que resuelvan el obligatorio de forma trivial.

## ¡Concurso!

Como en años anteriores, este año se realizará un concurso entre los trabajos presentados. **El concurso no influye en la nota y no es de participación obligatoria.** La mecánica del concurso consiste en que se pondrán los ejecutables en el sitio web de la asignatura, con un código de forma que nadie sepa quiénes son los autores (a menos que Uds. hagan aparecer sus nombres en la salida gráfica). Luego, al final del curso se habilitará una encuesta, donde todos los estudiantes de la asignatura podrán votar de forma anónima e individual a los tres mejores **juegos, dándole 3, 2 y 1 votos al primer, segundo y tercer juego votado, respectivamente.**

## Sugerencias

- Utilizar la documentación provista en [www.opengl.org](http://www.opengl.org)
- Examinar videos del juego para extraer ideas que aporten a lo gráfico.
- Intentar realizar en primera instancia una versión sencilla del juego que satisfaga los requerimientos mínimos y luego continuar con los extras y características más complejas.
- Utilizar el EVA para intercambiar opiniones sobre problemas, dudas, en lo que respecta a la letra, a la programación en OpenGL, etc.